Reporte Técnico de Actividades Práctico-Experimentales Nro. 2

1. Datos de Identificación del Estudiante y la Práctica

Nombre del estudiante(s)	- Bryan Troya - Anthony Yaguana - Darwin Correa - Derick Vargas
Asignatura	Sistemas Digitales
Ciclo	5
Unidad	1
Resultado de aprendizaje de la unidad	R1. Diseña arquitectura de microcontroladores, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica Nro.	2
Título de la Práctica	Juego de luces interactivo con ESP32
Nombre del Docente	Luis Darío Sinche C.
Fecha	23 de octubre de 2026
Horario	Viernes 11h30 a 13h30
Lugar	Laboratorio de computación (simulación en Wokwi y prueba en la placa ESP32 del grupo)
Tiempo planificado en el Sílabo	3 horas (APE, semana 2): 2 h presenciales y 1 h para terminar el informe
Modalidad de Trabajo	Cada estudiante simula en su computadora; se trabaja y se entrega en parejas; la prueba física se hace con la placa del grupo
Placa	ESP32 DevKit V1 de 30 pines (módulo ESP32-WROOM-32)

2. Objetivo(s) de la Práctica

- Programar en el ESP32 un juego de luces con cuatro LEDs cuya velocidad cambia con un pulsador, usando temporización no bloqueante con millis() y la lectura del pulsador con pull-up interna, antirrebote y detección de flanco.

3. Materiales, Reactivos, Equipos y Herramientas

- Documento de la Práctica de laboratorio Nro. 2
- Computadora
- Wokwi en el navegador
- Arduino IDE 2.X con el paquete **esp32 by Espressif Systems** versión 3.x, para la Parte C.
- Cuenta de GitHub y acceso al EVA, para la Parte D.
- Hardware (por grupo, para la parte C):

Cantidad	Componente
1	ESP32 DevKit V1 de 30 pines + cable USB de datos (no solo de carga)
1	Protoboard
4	LEDs de 5 mm rojos, amarillos o verdes (no azules ni blancos: casi no encienden a 3,3 V)
4	Resistencias de 330 Ω
1	Pulsador de 4 patas
6	Cables jumper macho-macho

4. Procedimiento / Metodología Ejecutada

- **Parte A:**

1. Se creó un nuevo proyecto de Wokwi de ESP32.
2. Se modificó la pestaña diagram.json cambiando su contenido por el del Anexo A.
3. Se verificó que las conexiones estén bien y coincidan con la Tabla de Conexiones y además se identificaron los pines RX2, TX2, D18, D19 y D21.

Componente	GPIO	Serigrafía en la placa	Conexión
LED 1	16	RX2	GPIO → resistencia 330 Ω → ánodo (pata larga); cátodo (pata corta) → GND
LED 2	17	TX2	Igual al LED 1
LED 3	18	D18	Igual al LED 1
LED 4	19	D19	Igual al LED 1
Pulsador	21	D21	Una pata al GPIO21 y la pata del lado

			opuesto a GND; sin resistencia externa (usa la pull-up interna)
Tierra	—	GND	Línea azul (−) de la protoboard; ahí van los cátodos y el pulsador

4. Se abrió la pestaña sketch.ino, se borró su contenido, se pegó el Código 1 y se presionó Start. Como resultado, se observó un solo LED encendido a la vez, avanzando en la secuencia 16 → 17 → 18 → 19 → 16… cada 0,3 s.
5. Se cambió el valor de PERIODO a 1000 y posteriormente a 100. Se ejecutó la simulación en cada ocasión y se completó la Tabla A.
6. Se volvió a configurar PERIODO = 300 y se tomó una captura de pantalla del circuito en funcionamiento.

- **Parte B:**

1. En el mismo proyecto, se reemplazó el contenido de sketch.ino por el Código 2 y se ejecutó la simulación.
2. Presionamos el pulsador varias veces y se obtuvieron los resultados esperados.

   Resultado obtenido: Cada clic cambia la velocidad y el Monitor Serie imprime el Periodo: ... ms una sola vez por clic.
3. Se presionó el pulsador durante 3 s (clic sostenido).

   Resultado obtenido: la velocidad cambia una sola vez, aunque el botón siga presionado.
4. Modificamos T_REBOTE a 0, ejecutamos, hicimos 10 clics y contamos cuántas líneas imprimió el Monitor Serie. Luego anotamos los resultados en la tabla B.
5. Volvimos a dejar T_REBOTE = 30, repetimos los 10 clics y completamos la Tabla B.
6. Tomamos una captura del Monitor Serie con T_REBOTE = 30.

- **Parte C:**

1. Con el USB desconectado, armamos el circuito en la protoboard siguiendo la Tabla de conexiones. Teniendo cuidado con la polaridad: pata larga del LED hacia la resistencia y el GPIO (pata corta) hacia GND.
2. Conectamos el cable USB de la ESP32 a la computadora. En Arduino IDE elegimos **Herramientas → Placa → ESP32 Dev Module** y el puerto detectado.
3. Cargamos el código 2 en Arduino IDE y presionamos **Subir**. EL código empezó a cargar correctamente en la ESP32.

4. Abrimos el Monitor Serie a 115200 baudios y repetimos los pasos B2 y B3 de la guía.

   Resultado obtenido: el mismo comportamiento que en la simulación en Wokwi.

5. Grabamos un video corto del montaje funcionando igual a lo simulado en Wokwi.

- **Parte D:**

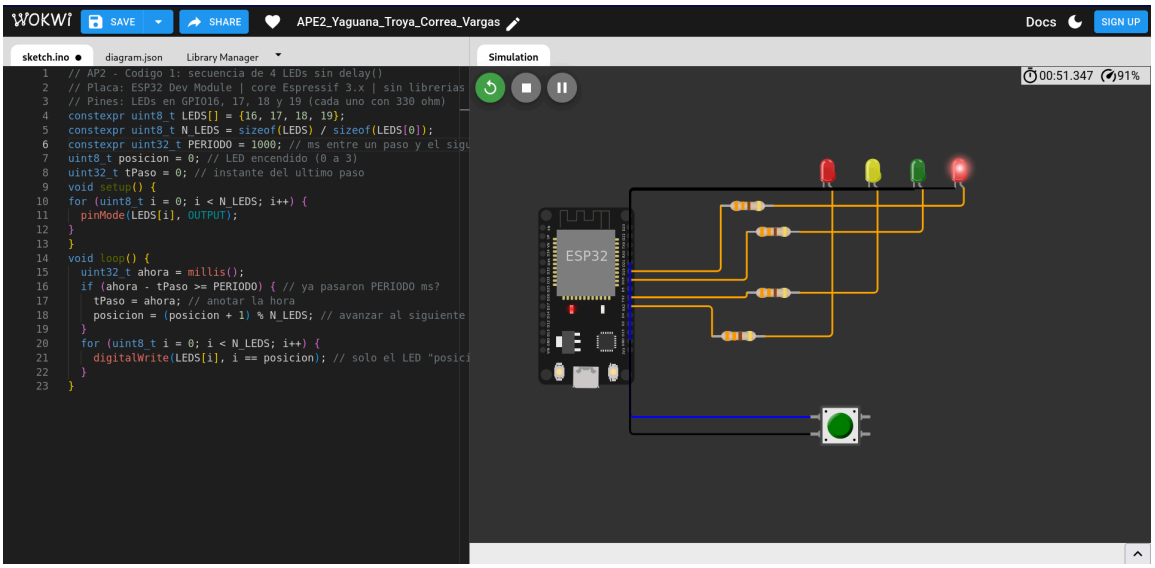
1. Creamos un repositorio en Github siguiendo la estructura indicada en la guía y subimos la práctica en la misma.

   Link del repositorio en Github: https://github.com/Anthony-YP/APE2-SistemasDigitales-Yaguana-Troya-Correa-vargas.git

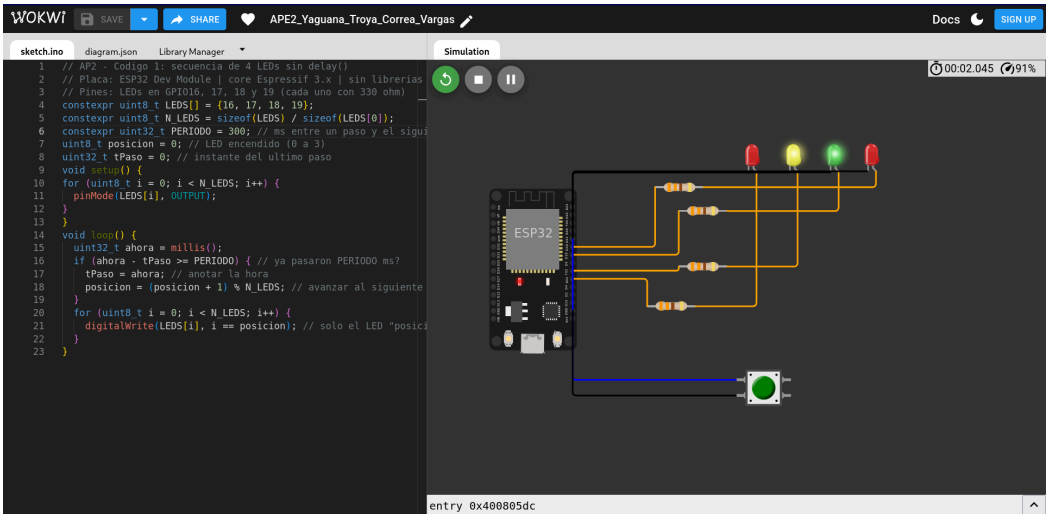
5. Resultados

- **Parte A**

Periodo 1000:



Periodo 300:



Periodo 100:

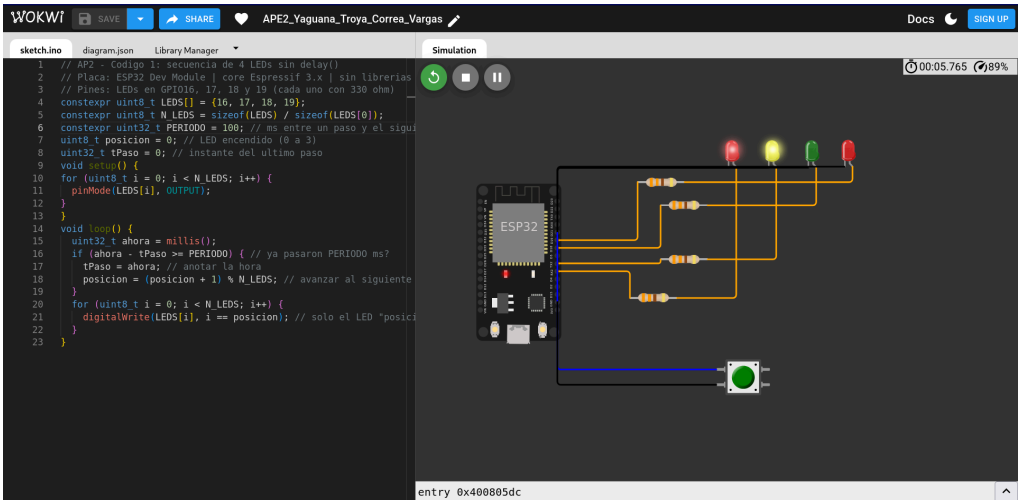
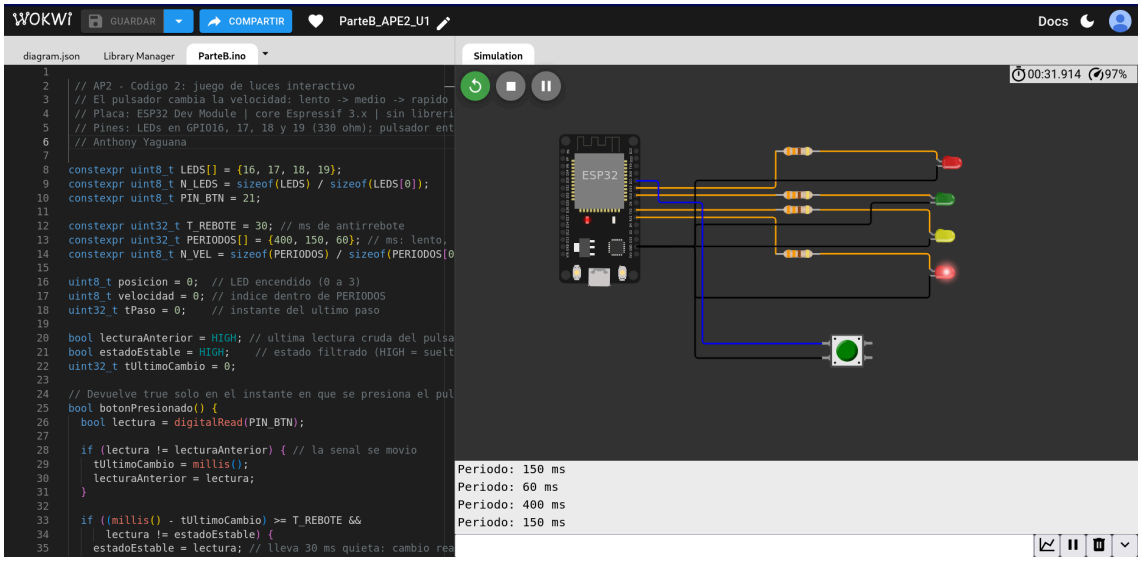
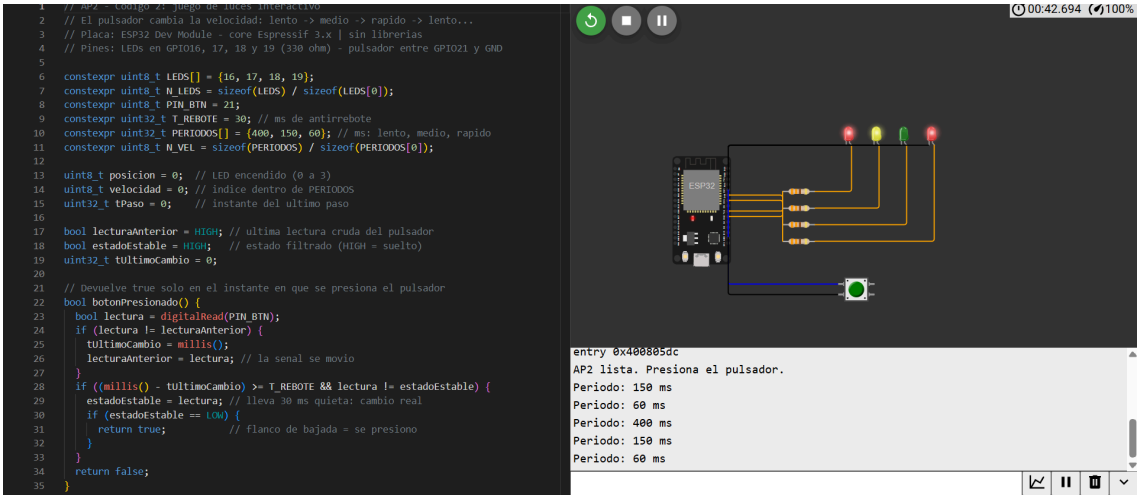


Tabla de Resultados:

PERIODO (ms)	¿Qué observan?
1000	un solo LED encendido a la vez, que avanza 16 → 17 → 18 → 19 → 16… cada 1 s.
300	un solo LED encendido a la vez, que avanza 16 → 17 → 18 → 19 → 16… cada 0,3 s.
100	un solo LED encendido a la vez, que avanza 16 → 17 → 18 → 19 → 16… cada 0,1 s.

- **Parte B**

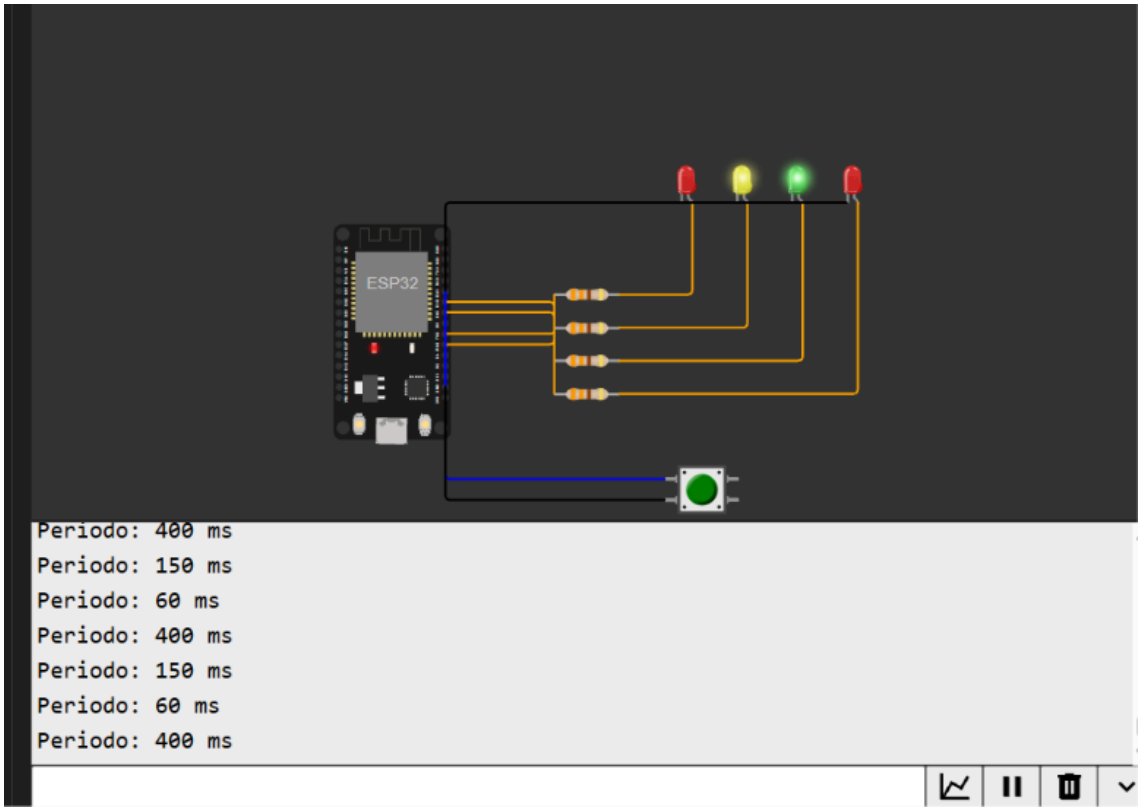




T_REBOTE = 0

```
// AP2 - Codigo 2: juego de luces interactivo
// El pulsador cambia la velocidad: lento -> medio -> rapido -> lento...
// Placa: ESP32 Dev Module | core Espressif 3.x | sin librerías
// Pines: LEDs en GPIO16, 17, 18 y 19 (330 ohm); pulsador entre GPIO21 y GND

constexpr uint8_t LEDS[] = {16, 17, 18, 19};
constexpr uint8_t N_LEDS = sizeof(LEDS) / sizeof(LEDS[0]);
constexpr uint8_t PIN_BTN = 21;
constexpr uint32_t T_REBOTE = 0;
constexpr uint32_t PERIODOS[] = {400, 150, 60}; // ms: lento, medio, rapido
constexpr uint8_t N_VEL = sizeof(PERIODOS) / sizeof(PERIODOS[0]);
```



Monitor Serie con T_REBOTE = 30.

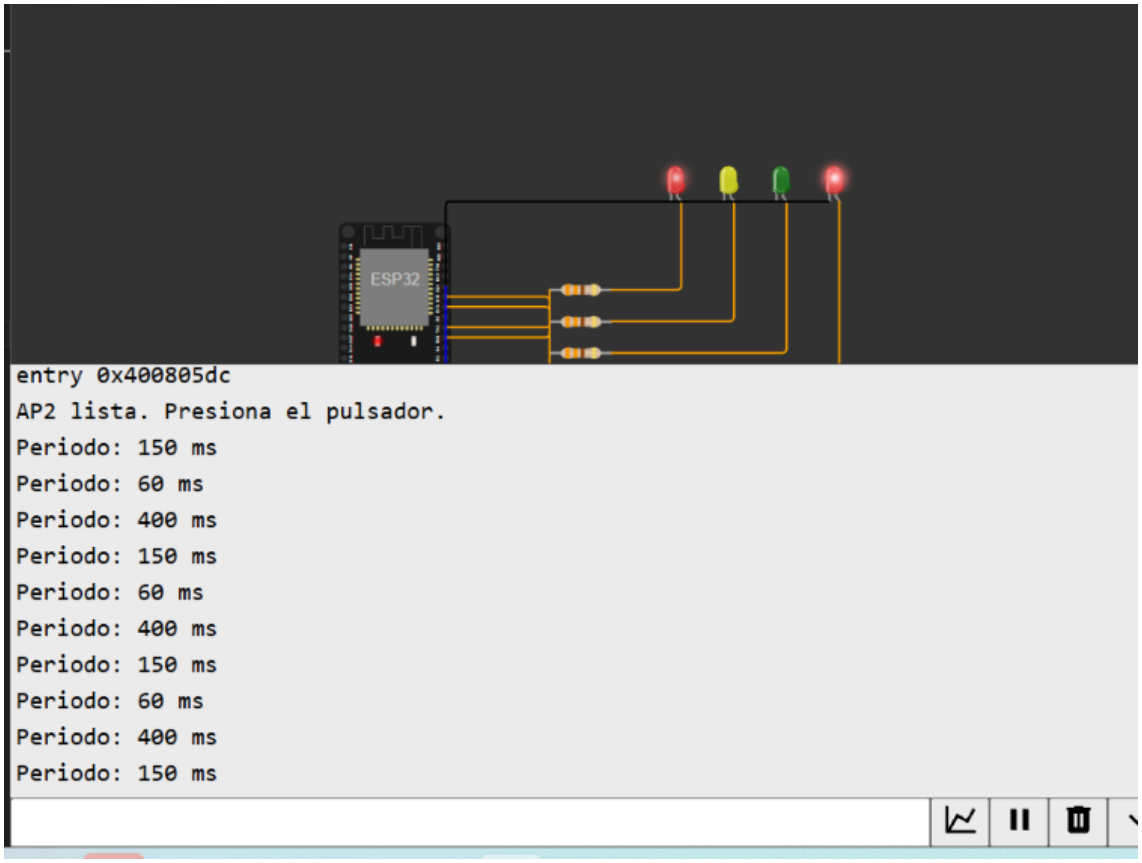
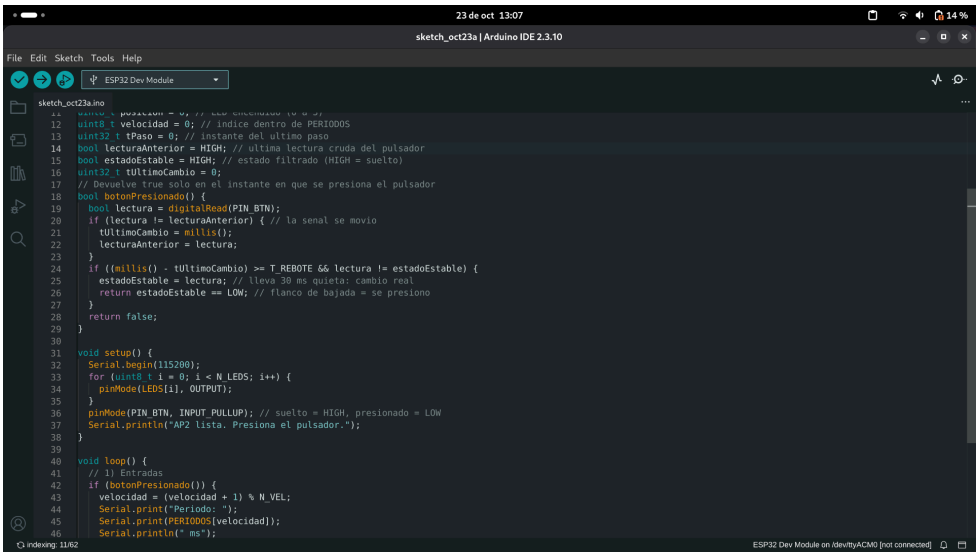


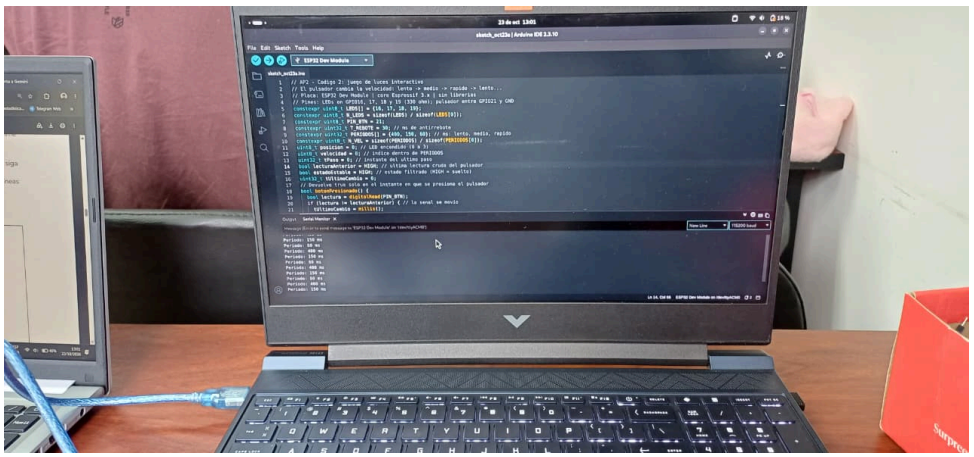
Tabla B: Efecto del antirrebote (10 clics):

T_REBOTE (ms)	Líneas impresas	¿Coincide con los 10 clics?
0	18	No
30	10	Si

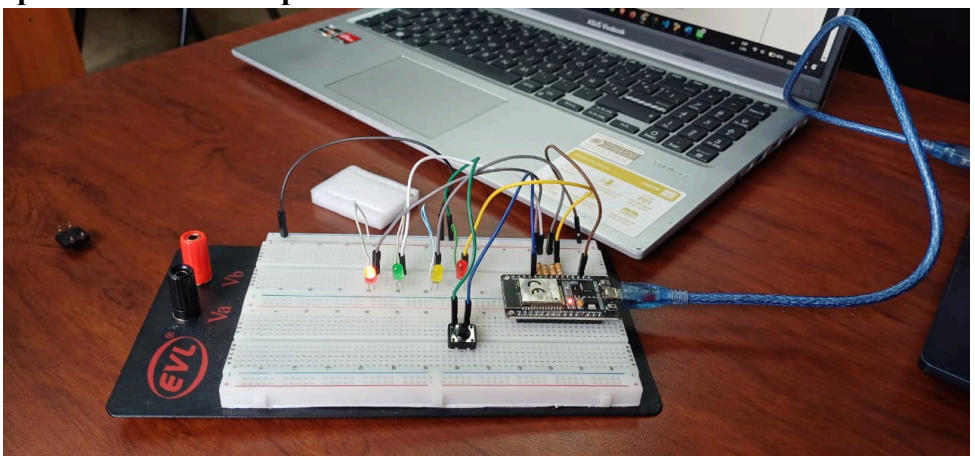
- **Parte C**

Sketch en Arduino IDE:





Implementación de la placa:



Video de la práctica: APE2.mp4

- **Parte D**

  Link del repositorio en Github: https://github.com/Anthony-YP/APE2-SistemasDigitales-Yaguana-Troya-Correa-vargas.git

6. Preguntas de Control

- **¿Por qué el Código 1 usa millis() en lugar de delay()? ¿Qué pasaría con el pulsador del Código 2 si cada paso de los LEDs usara delay(400)?**

  Porque millis() permite hacer temporización no bloqueante. Si se usara delay(400), el programa se detendría por completo 400 ms y no detectaría el botón si lo presionas en ese lapso.

- **El pulsador va entre el GPIO21 y GND, sin resistencia externa. ¿Qué valor lee digitalRead() con el pulsador suelto y con el pulsador presionado? ¿Por qué?**

  Suelto lee HIGH gracias a la resistencia pull-up interna configurada en el pin. Presionado lee LOW porque el pulsador lo conecta directamente a tierra (GND).

- **¿Qué pasaría si conectaran el pulsador al GPIO34 sin cambiar el código? Justifiquen con lo visto en clase.**

  No funcionaría porque el pin 34 pertenece a los pines que son de "solo entrada" (34-39), los cuales no cuentan con resistencia pull-up interna.

- **En el paso B3 la velocidad cambió una sola vez aunque mantuvieron presionado el pulsador. ¿Qué parte del Código 2 lo garantiza?**

  La detección de flanco en la función botonPresionado(). Específicamente el bloque if (estadoEstable == LOW) { return true; }, que ejecuta la acción una sola vez al detectar la bajada de señal.

- **Expliquen la Tabla B: ¿por qué con T_REBOTE = 0 puede haber más líneas impresas que clics?**

Al poner T_REBOTE = 0 se desactiva el antirrebote por software. Esto hace que el microcontrolador cuente los rebotes mecánicos de los contactos del pulsador como múltiples clics muy rápidos.

- **Calculen la corriente que circula por un LED rojo (Vf ≈ 2,0 V) con una resistencia de 330 Ω a 3,3 V. ¿Por qué no conviene usar un LED azul?**

  Corriente = (3,3 V - 2,0 V) / 330 ohmios = 3,94 mA. No conviene usar LEDs azules o blancos porque necesitan más voltaje y casi no encienden a 3,3 V.
- **Mencionen una diferencia entre probar el circuito en Wokwi y probarlo en la placa real.**

  En la placa real, debemos tener cuidado con la polaridad de los leds , la calidad de los cables jumper, las corrientes y posibles malas conexiones en la protoboard.
- **El ESP32 sigue funcionando a 240 MHz aunque el juego solo cambie de LED cada 400 ms. Propongan una idea para reducir el consumo de energía si el juego funcionara con batería.**

  Activar un modo de bajo consumo de energía (como el Light Sleep del ESP32) durante los tiempos de espera entre cada cambio de LED para pausar la CPU.

7. Conclusiones

- La temporización no bloqueante con millis() es fundamental para sistemas interactivos, ya que a diferencia de delay(), que congela la ejecución del microcontrolador, permite que el bucle principal siga atendiendo otras tareas como la lectura del pulsador mientras se controla el tiempo entre pasos de la secuencia de LEDs, garantizando una respuesta inmediata a las entradas del usuario.
- La resistencia pull-up interna simplifica el hardware sin sacrificar funcionalidad, ya que configurar el pin del pulsador con INPUT_PULLUP permite conectarlo directamente entre el GPIO y GND, eliminando la necesidad de una resistencia externa y reduciendo componentes, cableado y posibles fallas, manteniendo una lectura digital clara de HIGH en reposo y LOW al presionar.
- El antirrebote por software y la detección de flanco son indispensables para una interacción confiable, porque los contactos mecánicos del pulsador generan rebotes durante varios milisegundos y sin el filtrado temporal y la detección de flanco de bajada una sola pulsación podría registrarse como múltiples eventos, como se evidenció en la Tabla B con 18 líneas impresas con T_REBOTE en 0 frente a 10 con T_REBOTE en 30, además de que la detección de flanco garantiza que mantener presionado el botón no genere cambios repetidos de velocidad.
- La elección adecuada de pines en el ESP32 es crítica para evitar comportamientos erráticos, ya que se deben evitar los pines de strapping, los pines solo de entrada que carecen de pull-up interna y los pines del USB, y además es necesario verificar la serigrafía de la placa porque los GPIO 16 y 17 aparecen etiquetados como RX2 y TX2, lo que puede inducir a errores de conexión.

- La simulación en Wokwi es una herramienta válida y eficaz para validar el diseño antes de la implementación física, ya que las capturas y resultados obtenidos coincidieron con el comportamiento esperado y con la prueba en la placa real, lo que confirma que la simulación permite depurar la lógica del código, verificar el cableado y completar las tablas de resultados sin necesidad de hardware, reduciendo costos y riesgos antes de la puesta en marcha con componentes reales

8. Recomendaciones

- Evitar pines problemáticos del ESP32 para futuros diseños, es una buena práctica continuar evitando los pines de strapping (0, 2, 5, 12, 15), los pines que solo funcionan como entrada (34 al 39) y los pines del puerto USB (1 y 3), ya que pueden causar comportamientos erráticos durante el arranque o limitar la funcionalidad del circuito.
- Medir las resistencias antes de empezar a armar el circuito para evitar resistencias desvalorizadas.
- Se recomienda primero realizar el diseño y la simulación del circuito en herramientas virtuales como Wokwi antes de armarlo en la protoboard. Esto permite verificar la lógica del código, comprobar el correcto funcionamiento de las conexiones y detectar posibles errores de polaridad o cortocircuitos sin ningún riesgo.

9. Declaración de uso de IA

Para el desarrollo y redacción del presente informe correspondiente a la Práctica Nro. 2, se declara el uso de la herramienta de inteligencia artificial generativa Chat GPT. Su asistencia se estructuró principalmente en la siguiente área:

- **Comprensión teórica y análisis de código:** Se utilizó como apoyo interactivo para entender a mayor profundidad la lógica de programación en C++, específicamente el funcionamiento de la temporización no bloqueante mediante la función millis() y la implementación estructural del filtro de antirrebote por software para el pulsador.

10. Bibliografía / Referencias

- Guía del APE Nro. 2

11. Anexos

- **Documento en línea:** APE2.docx
- **Wokwi Bryan Troya:**
  Parte A: https://wokwi.com/projects/478691171356978177
  Parte B: https://wokwi.com/projects/478728451474355201
- **Wokwi Anthony Yaguana:**
  Parte A: https://wokwi.com/projects/478729000076829697
  Parte B: https://wokwi.com/projects/478729709744792577
- **Wokwi Derick Vargas:**
  Parte A: https://wokwi.com/projects/478692101111713793
  Parte B: https://wokwi.com/projects/478692101111713793
- **Wockwi Darwin Correa:**
  Parte A:https://wokwi.com/projects/478728415649747969
  Parte B:https://wokwi.com/projects/478729302016445441
- **Video de la práctica:** APE2.mp4
- **Link del repositorio:** https://github.com/Anthony-YP/APE2-SistemasDigitales-Yaguana-Troya-Correa-vargas.git